

Um *Chat* com *Sockets* UDP ou TCP

Prof. Msc. Emerson Assis de Carvalho
emerson.carvalho@ifsuldeminas.edu.br

Sistemas Computacionais Distribuídos

Instituto Federal do Sul de Minas Gerais - Campus Machado

1 Introdução

Esta lista apresenta a versão final de um *chat* desenvolvido com *sockets* UDP ou TCP em *Java*. Mais sobre a *networking API* do *Java* pode ser encontrada em [1].

2 Interfaces *Sender* e *Receiver*

As interfaces *Sender* e *Receiver* representam o que uma unidade do programa de *chat* precisa ter. Precisa de uma forma para **enviar** mensagens e uma forma para **receber** mensagens.

Listagem 1. Interface *Sender*

```
package br.edu.ifsuldeminas.mch.sd.chat;

public interface Sender {
    void send(String message) throws ChatException;
}
```

Listagem 2. Interface *Receiver*

```
package br.edu.ifsuldeminas.mch.sd.chat;

public interface Receiver extends Runnable {
    void run();
}
```

Aqui vale ressaltar o fato da interface *Receiver* estender *Runnable*, que vai nos permitir colocar o recebimento de mensagens em uma *thread* separada das funções de envio. Para termos interatividade com o *chat*, essas duas funções, envio e recebimento, precisam rodar em paralelo.

3 Encapsulamento de erros

Essa classe de exceção permite que o tratamento dos erros, por parte dos usuários das implementações das interfaces de envio e recebimento, seja feito de forma mais limpa. Essa será a única exceção com a qual os usuário precisara se preocupar.

Listagem 3. Classe de erro que encapsula diferentes erros

```
package br.edu.ifsuldeminas.mch.sd.chat;
```

```

public class ChatException extends Exception {

    private static final long serialVersionUID = 1L;

    public ChatException(String message, Throwable cause) {
        super(message, cause);
    }
}

```

4 Exibição das mensagens

A interface *MessageContainer* servirá para determinar onde as mensagens recebidas serão apresentadas.

Listagem 4. Interface representando um *container* de mensagens

```

package br.edu.ifsuldeminas.mch.sd.chat;

public interface MessageContainer {
    void newMessage(String message);
}

```

5 *Sender* e *Receiver* UDP

A implementação UDP da interface *Sender* faz uso de *sockets* UDP para realizar o envio de mensagens. Na criação da classe é necessário informar o receptor e sua porta. O envio de mensagens pode ser feito pelo método *send(message)*.

Listagem 5. *Sender* UDP

```

package br.edu.ifsuldeminas.mch.sd.chat;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.net.SocketException;

class UDPSender implements Sender {
    private DatagramSocket senderSocket = null;
    private InetAddress receiver;
    private int receiverPort;

    public UDPSender(InetAddress receiver, int receiverPort)
        throws ChatException {

        validateAttributes(receiver, receiverPort);
        this.receiver = receiver;
        this.receiverPort = receiverPort;

        try {
            this.senderSocket = new DatagramSocket();
        } catch (SocketException socketException) {
            throw new ChatException(
                "There was some errors starting your sender.",
                socketException);
        }
    }

    public void send(String message) throws ChatException {
        byte[] messageBytes = message.getBytes();

        DatagramPacket datagramPacketToSend = new DatagramPacket(messageBytes,
            messageBytes.length, receiver, receiverPort);
    }
}

```

```

    try {
        senderSocket.send(datagramPacketToSend);
    } catch (IOException ioException) {
        throw new ChatException(String.format(
            "Your message '%s' can't be sent.", message), ioException);
    }
}

private void validateAttributes(InetAddress receiver, int receiverPort) {
    if (receiver == null)
        throw new IllegalArgumentException("Receiver can't be null.");

    if (receiverPort <= 0)
        throw new IllegalArgumentException(
            String.format("Invalid port number for receiver."));
}
}

```

A implementação UDP da interface *Receiver* faz uso de *sockets* UDP para realizar o recebimento de mensagens. Na criação da classe é necessário informar a porta onde irá receber as mensagens e o container que irá exibi-las. O recebimento das mensagens em uma *thread* é feita de forma transparente para o usuário da classe.

Listagem 6. Receiver UDP

```

package br.edu.ifsuldeminas.mch.sd.chat;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.SocketException;

class UDPReceiver implements Receiver {

    private static int MIN_BUFFER_SIZE = 1000;
    private static int MIN_PORT_NUMBER = 1024;

    private int portNumber;
    private int bufferSize;
    private DatagramSocket receiverSocket = null;
    private byte[] incomingBuffer = null;
    private boolean isRunning = false;

    private MessageContainer container;

    public UDPReceiver(int portNumber, int bufferSize,
        MessageContainer container) throws ChatException {
        validateAttributes(portNumber, bufferSize, container);
        assignAttributes(portNumber, bufferSize, container);

        try {
            prepare();
        } catch (SocketException socketException) {
            throw new ChatException(
                "There was some errors starting yor receiver.",
                socketException);
        }

        new Thread(this).start();
    }

    public void run() {
        if (!isRunning) {
            isRunning = true;

            while (true) {

```

```

        try {
            receive();
        } catch (IOException ioException) {
            container
                .newMessage("There was some errors when receiving messages.");
        }
    }
}

private void validateAttributes(int portNumber, int bufferSize,
    MessageContainer container) {
    if (portNumber <= MIN_PORT_NUMBER)
        throw new IllegalArgumentException(
            "The Receiver can't use reserved ports.");

    if (container == null)
        throw new IllegalArgumentException(
            "The message container can't be null.");

    if (bufferSize < MIN_BUFFER_SIZE)
        throw new IllegalArgumentException(String.format(
            "The buffers size needs to be greater than %d",
            MIN_BUFFER_SIZE));
}

private void assignAttributes(int portNumber, int bufferSize,
    MessageContainer container) {
    this.portNumber = portNumber;
    this.bufferSize = bufferSize;
    this.container = container;
}

private void prepare() throws SocketException {
    receiverSocket = new DatagramSocket(portNumber);
}

private void receive() throws IOException {
    incomingBuffer = new byte[bufferSize];
    DatagramPacket received = new DatagramPacket(incomingBuffer,
        incomingBuffer.length);

    receiverSocket.receive(received);

    container.newMessage(new String(received.getData()));
}
}

```

6 *Sender e Receiver TCP*

A implementação TCP da interface *Sender* faz uso de *sockets* TCP para realizar o envio de mensagens. Na criação da classe é necessário informar o receptor e sua porta. O envio de mensagens pode ser feito pelo método *send(message)*.

Listagem 7. *Sender TCP*

```

package br.edu.ifsuldeminas.mch.sd.chat;

import java.io.DataOutputStream;
import java.io.IOException;
import java.net.ConnectException;
import java.net.InetAddress;
import java.net.Socket;

public class TCPSender implements Sender {

```

```

private Socket socket = null;
private DataOutputStream outputFlow;

private InetAddress receiver;

public TCPSender(InetAddress receiver, int receiverPort)
    throws ChatException {

    this.receiver = receiver;

    try {
        boolean isConnected = false;
        while (!isConnected) {
            try {
                this.socket = new Socket(this.receiver, receiverPort);
                isConnected = true;
            } catch (ConnectException connectException) {
                // Here can treat max connection attempts
            }
        }

        this.outputFlow = new DataOutputStream(
            this.socket.getOutputStream());
    } catch (IOException ioException) {
        throw new ChatException(
            "There was some errors starting your sender.", ioException);
    }
}

public void send(String message) throws ChatException {
    try {
        this.outputFlow.writeUTF(message);
    } catch (IOException ioException) {
        throw new ChatException("There was some sending your message",
            ioException);
    }
}
}

```

A implementação TCP da interface *Receiver* faz uso de *sockets* TCP para realizar o recebimento de mensagens. Na criação da classe é necessário informar a porta onde irá receber as mensagens e o container que irá exibi-las. O recebimento das mensagens em uma *thread* é feita de forma transparente para o usuário da classe.

Listagem 8. *Receiver* TCP

```

package br.edu.ifsuldeminas.mch.sd.chat;

import java.io.DataInputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class TCPReceiver implements Receiver {

    private int portNumber;
    private MessageContainer container;

    private ServerSocket receiverSocket;
    private Socket socket;
    private DataInputStream inputFlow;

    public TCPReceiver(int portNumber, MessageContainer container)
        throws ChatException {
        this.portNumber = portNumber;
        this.container = container;
    }
}

```

```

    try {
        prepare();
    } catch (IOException ioException) {
        throw new ChatException(
            "There was some errors starting yor receiver.", ioException);
    }

    new Thread(this).start();
}

private void prepare() throws IOException {
    this.receiverSocket = new ServerSocket(this.portNumber);
}

public void run() {
    try {
        this.socket = this.receiverSocket.accept();
        this.inputFlow = new DataInputStream(this.socket.getInputStream());

        while (true) {
            receive();
        }
    } catch (IOException ioException) {
        container
            .newMessage("There were some errors when receiving messages.");
    }
}

private void receive() throws IOException {
    String message = this.inputFlow.readUTF();

    this.container.newMessage(message);
}
}

```

7 Fábrica de *chat*

Essa classe cria *Senders* e *Receivers*. Encapsula a implementação de como é feito o envio e recebimento de mensagens. Por meio dessa classe, os clientes do *chat* apenas especificam se desejam uma implementação orientada ou não à conexão. Oculta detalhes sobre a criação do ambiente e nos deixa a vontade para mudar esse cenário. Podemos, por exemplo, usar *sockets* TCP, *sockets* UDP ou outra forma de passagem de mensagens.

Listagem 9. Fábrica de *chat*

```

package br.edu.ifsuldeminas.mch.sd.chat;

import java.net.InetAddress;
import java.net.UnknownHostException;

public class ChatFactory {

    private static int DEFAULT_RECEIVER_BUFFER_SIZE = 1000;
    private static String serverName;
    private static int serverPort;
    private static int localPort;
    private static MessageContainer container;

    public static Sender build(boolean isConnectionOriented, String serverName,
        int serverPort, int localPort, MessageContainer container)
        throws ChatException {
        ChatFactory.serverName = serverName;
        ChatFactory.serverPort = serverPort;
        ChatFactory.localPort = localPort;
    }
}

```

```

        ChatFactory.container = container;

        return build(isConnectionOriented);
    }

    private static Sender build(boolean isConnectionOriented)
        throws ChatException {
        try {
            // TCP
            if (isConnectionOriented) {
                new TCPReceiver(ChatFactory.localPort, ChatFactory.container);

                return new TCPSender(InetAddress.getByName(ChatFactory.serverName),
                    ChatFactory.serverPort);
            }
            // UDP
        } else {
            new UDPReceiver(ChatFactory.localPort,
                DEFAULT_RECEIVER_BUFFER_SIZE, ChatFactory.container);

            return new UDPSender(InetAddress.getByName(ChatFactory.serverName),
                ChatFactory.serverPort);
        }
    } catch (UnknownHostException unknownHostException) {
        throw new ChatException("The receiver is unknown",
            unknownHostException);
    }
}

```

8 Cliente

Para o cliente basta cuidar da entrada (leitura do teclado, por exemplo) e saída (implementação do *container* de mensagens) para os usuários, além de usar a fábrica para criar o ambiente do *chat*.

Uma implementação da interface *MessageContainer*, a classe *SysOutContainer*, apresenta as mensagens recebidas no *console* padrão de saída. Repare que essa classe já faz parte do cliente do *chat*, que pode determinar diferentes lugares para a exibição das suas mensagens.

Listagem 10. Container para saída no console

```

package br.edu.ifsuldeminas.mch.sd.chat.client;

import br.edu.ifsuldeminas.mch.sd.chat.MessageContainer;

public class SysOutContainer implements MessageContainer {
    public void newMessage(String message) {
        System.out.println(String.format("> %s", message));
    }
}

```

A entrada de mensagens também fica a critério do cliente e, nesse caso, foi feita pelo próprio console no *main* da aplicação.

Listagem 11. Container para saída no console

```

package br.edu.ifsuldeminas.mch.sd.chat.client;

import java.util.Scanner;

import br.edu.ifsuldeminas.mch.sd.chat.ChatException;
import br.edu.ifsuldeminas.mch.sd.chat.ChatFactory;
import br.edu.ifsuldeminas.mch.sd.chat.Sender;

public class Chat {

```

```

public static String KEY_TO_EXIT = "q";
public static int RECEIVER_BUFFER_SIZE = 1000;

public static void main(String[] args) {
    Scanner reader = null;

    int localPort = 0;
    int serverPort = 0;

    try {
        localPort = Integer.parseInt(args[0]);
        serverPort = Integer.parseInt(args[1]);
    } catch (NumberFormatException e1) {
        System.err.printf("Errors on input parameters");

        System.exit(1);
    }

    try {
        Sender sender = ChatFactory.build(true, "localhost", serverPort,
            localPort, new SysOutContainer());

        reader = new Scanner(System.in);
        String message = "";
        while (!message.equals(KEY_TO_EXIT)) {
            message = reader.nextLine();
            if (!message.equals("q")) {
                sender.send(message);
            } else
                System.exit(0);
        }
    } catch (NumberFormatException e) {
        e.printStackTrace();
    } catch (ChatException chatException) {
        System.err.printf(String.format(
            "There was a problem with the chat. Error message: %s",
            chatException.getCause().getMessage()));

        reader.close();
        System.exit(0);
    }
}
}

```

9 Melhorias

Podemos fazer algumas melhorias no cliente e no próprio *chat*, no que se refere a API e as implementações. Vamos começar implementando um cliente visual, ou seja, usar o *chat* para criar uma aplicação *swing*. Se for necessário, adaptações no *chat* poderão ser realizadas para melhorar a usabilidade por parte dos clientes.

Referências

- [1] Oracle. Hierarchy for package java.net. <http://docs.oracle.com/javase/7/docs/api/java/net/package-tree.html>, 2013. Acessado em 01/09/2013.